

AUTOMATED DEPLOYMENT OF A STATIC WEBSITE TO AWS EC2

USING GITHUB ACTIONS (CI/CD)

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Juttuka Lovababu, APSSDC

Submitted by

TADIKONDA YASASWINI, 23BQ1A42G9, VVIT NAMBUR,

SHAIK ABDUL KAREEM, 23BQ1A42F9, VVIT NAMBUR,

SHAIK MOMINA, 23BQ1A42G4, VVIT NAMBUR,

SHAIK SHAHED ALI, 23BQ1A42G5, VVIT NAMBUR,

SHAIK SHAZIA, 23BQ1A42G6, VVIT NAMBUR,

THOTA GAYATHRI, 23BQA1A42H5, VVIT NAMBUR,

YELE HARSHITHA, 23BQ1A42I9, VVIT NAMBUR.

SHAIK BAJI VALI, 23BQ1A42G1, VVIT NAMBUR

TALLURI YOGESH CHANDRA, 23BQ1A42H0, VVIT NAMBUR

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr Juttuka Lovababu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Tadikonda Yaraswini, 23BQ1A42G9, yaraswinitadikonda@gmail.com
Shaik Abdul Kareem, 23BQ1A42F9, alkareemshaik027@gmail.com
Shaik Momina, 23BQ1A42G4, momina.shaik03@gmail.com
Shaik Shahed Ali, 23BQ1A42G5, shahedshaik363@gmail.com
Shaik Shazia, 23BQ1A42G6, shaziashaik2005@gmail.com
Thota Gayathri, 23BQ1A42H5, thotagayathri222@gmail.com
Yele Harshitha, 23BQ1A42I9, harshithayele2005@gmail.com
Talluri Yogesh Chandra, 23BQ1A42H0, talluriyogeshchandra@gmail.com
Shaik Baji Vali, 23BQ1A42G1, bajivali269@gmail.com

ABSTRACT

Continuous Integration and Continuous Deployment (CI/CD) have become essential practices in modern software development, enabling faster, more reliable, and automated application delivery. This project presents the design and implementation of an automated deployment pipeline for a static hospital website using GitHub Actions and Amazon Web Services (AWS) Elastic Compute Cloud (EC2). The primary objective is to eliminate manual deployment tasks and establish a seamless workflow that automatically deploys website updates whenever changes are pushed to the GitHub repository.

The project integrates Git for version control, GitHub as the source code repository, GitHub Actions for workflow automation, AWS EC2 as the cloud hosting platform, Apache HTTP Server for web hosting, and SSH/SCP protocols for secure communication and file transfer. Whenever a developer commits and pushes code to the main branch, GitHub Actions automatically triggers the deployment workflow, securely transfers the updated website files to the EC2 instance, replaces the existing files in the Apache web server's document root, and restarts the web server. This fully automated process ensures that the latest version of the website is deployed without any manual intervention.

The proposed solution improves deployment speed, consistency, and reliability while minimizing human errors commonly associated with manual deployments. It also provides practical exposure to industry-standard DevOps tools and cloud technologies, making it highly relevant for modern software engineering practices. The successful implementation of this project demonstrates how CI/CD pipelines can streamline web application deployment, enhance development efficiency, and provide a scalable foundation for future cloud-native applications.

Furthermore, the project highlights the practical implementation of DevOps principles by integrating version control, automation, cloud computing, and secure deployment into a unified workflow. The automated pipeline not only reduces deployment time but also simplifies the software release process, allowing developers to focus on application development rather than repetitive deployment tasks. The architecture is designed to be scalable and can be extended to support advanced features such as containerization with Docker, orchestration using Kubernetes, HTTPS with SSL/TLS, load balancing, auto-scaling, and monitoring services. This makes the proposed solution a strong foundation for building robust, secure, and production-ready web applications in modern cloud environments.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	Introduction	5
2.	Methodology	7
3.	Technology Stack	10
4.	System Design/Architecture	13
5.	Implementation	19
6.	Results	25
7.	Conclusion	29

1. INTRODUCTION

In today's fast-paced software development environment, organizations strive to deliver applications quickly, reliably, and with minimal downtime. Traditional deployment methods often involve manually transferring application files to servers, configuring services, and restarting web servers. These manual processes are time-consuming, prone to human error, and difficult to maintain as applications grow in complexity. To overcome these challenges, modern software development has widely adopted DevOps practices, particularly Continuous Integration (CI) and Continuous Deployment (CD), which automate the software development and deployment lifecycle.

Continuous Integration (CI) is the practice of frequently integrating code changes into a shared repository, where automated processes validate and prepare the application for deployment. Continuous Deployment (CD) extends this concept by automatically deploying validated changes to the target environment without requiring manual intervention. Together, CI/CD improves software quality, accelerates release cycles, enhances collaboration among development teams.

This project focuses on implementing an automated deployment pipeline for a static hospital website using GitHub Actions and Amazon Web Services (AWS) Elastic Compute Cloud (EC2). The website is developed using HTML, CSS, and JavaScript, while Git is used for version control and GitHub serves as the central source code repository. GitHub Actions acts as the automation platform that continuously monitors the repository for code changes. Whenever a developer pushes updates to the main branch, a predefined workflow is triggered automatically. The workflow securely authenticates with the AWS EC2 instance using SSH, transfers the updated website files using the Secure Copy Protocol (SCP), deploys them to the Apache web server's document root, and restarts the Apache service, making the latest version of the website immediately available to users.

AWS EC2 provides a scalable and reliable cloud computing environment for hosting the website, while Apache HTTP Server efficiently serves the web pages to clients over the internet. Secure authentication using SSH keys and GitHub Secrets ensures that

deployment credentials remain protected throughout the automation process. This integration of cloud infrastructure, version control, automation tools.

The proposed system significantly reduces deployment time, eliminates repetitive manual tasks, and minimizes deployment failures caused by human errors. In addition to improving operational efficiency, the project provides hands-on experience with industry-standard technologies such as Git, GitHub, GitHub Actions, AWS EC2, Apache, SSH, and SCP. These technologies are widely adopted in software development organizations, making this project highly relevant for understanding real-world CI/CD workflows.

Overall, this project demonstrates how automated deployment pipelines can simplify website deployment, improve software delivery, and establish a reliable foundation for scalable cloud-based applications. The implementation serves as an effective example of integrating DevOps methodologies with cloud computing to achieve secure, efficient, and continuous software deployment.

2. METHODOLOGY

The methodology adopted for this project follows a systematic DevOps approach to automate the deployment of a static hospital website using GitHub Actions and AWS EC2. The implementation consists of several sequential stages, beginning with website development and ending with the automatic deployment of the updated website to the cloud server. Each stage contributes to creating a reliable, secure, and efficient Continuous Integration and Continuous Deployment (CI/CD) pipeline.

1. Requirements Gathering

The project begins with the development of a static hospital website using HTML, CSS, and JavaScript. The website consists of multiple responsive web pages that provide essential information such as hospital services, departments, doctors, contact details, and appointment-related content. The source code is developed and maintained using Visual Studio Code and is organized in a structured project directory.

2. Version Control using Git

Git is used as the version control system to manage the source code efficiently. Developers track changes, maintain different versions of the project, and collaborate without affecting the main codebase. After implementing new features or modifications, the updated files are committed to the local Git repository before being pushed to GitHub.

3. Source Code Management with GitHub

The GitHub repository serves as the centralized platform for storing the project source code. It enables secure code management, version history, and collaboration. Every code push to the main branch acts as the trigger for initiating the automated deployment workflow through GitHub Actions.

4. CI/CD Workflow using GitHub Actions

GitHub Actions is configured to automate the deployment process. A workflow file written in YAML is placed inside the .github/workflows directory of the repository. Whenever changes are pushed to the main branch, GitHub Actions automatically executes the workflow, eliminating the need for manual deployment. The workflow

performs tasks such as checking out the latest source code, establishing a secure connection with the AWS EC2 instance, transferring the updated website files, and executing deployment commands on the server.

5. Secure Authentication and File Transfer

To ensure secure communication between GitHub and the AWS server, SSH key authentication is implemented. Sensitive information such as the EC2 host address, username, and private SSH key is securely stored using GitHub Secrets. During workflow execution, GitHub Actions retrieves these credentials, establishes a secure SSH connection, and transfers the website files to the EC2 instance using the Secure Copy Protocol (SCP).

6. Deployment on AWS EC2

An AWS EC2 instance running the Ubuntu operating system is configured as the hosting environment. Apache HTTP Server is installed and configured to serve the website. The transferred website files replace the existing files in the Apache document root (`/var/www/html`). After deployment, the Apache service is restarted to ensure that the latest version of the website is immediately accessible through the server's public IP address.

7. Verification and Testing

After each deployment, the website is verified by accessing the EC2 instance through a web browser. Functional testing is performed to ensure that all pages load correctly and that the latest updates are reflected successfully. GitHub Actions workflow logs are monitored to confirm successful execution of each deployment stage and to identify any issues during the automation process.

8. Continuous Deployment Process

The implemented CI/CD pipeline enables continuous deployment of the website. Any future modifications require only updating the source code and pushing the changes to the GitHub repository. The complete deployment process—including workflow execution, secure file transfer, server update, and Apache restart—is performed automatically without manual intervention. This approach significantly reduces deployment time, improves consistency, minimizes human errors, and provides a scalable deployment solution aligned with modern DevOps practices.

The overall methodology demonstrates the integration of version control, workflow automation, cloud infrastructure, and secure deployment mechanisms to create an efficient and reliable software deployment pipeline. The implemented solution ensures faster software delivery while maintaining consistency, security, and ease of maintenance throughout the deployment lifecycle.

3. TECHNOLOGY STACK

The technology stack selected for this project was carefully chosen to provide an efficient, secure, and automated deployment solution while following modern DevOps practices. The integration of these technologies enables seamless communication between the development environment, source code repository, automation platform, cloud infrastructure, and web server.

The development process begins in Visual Studio Code, where the website source code is created and modified using HTML, CSS, and JavaScript. Git is used to monitor every change made to the project files, allowing developers to maintain version history and collaborate effectively. The source code is then pushed to the GitHub repository, which serves as the centralized platform for project management and code storage.

Once code is pushed to the main branch, GitHub Actions automatically detects the changes and initiates the CI/CD workflow. The workflow authenticates with the AWS EC2 instance through SSH using encrypted credentials stored securely as GitHub Secrets. The latest website files are transferred using the Secure Copy Protocol (SCP), after which deployment commands are executed remotely to replace the previous website files and restart the Apache HTTP Server. This automated integration eliminates manual deployment activities and significantly improves deployment efficiency.

Benefits of this Technology Stack

The selected technology stack offers several advantages that make it suitable for implementing an automated deployment pipeline:

- **Automation:** GitHub Actions automates repetitive deployment tasks, reducing manual effort and improving productivity.
- **Version Control:** Git enables efficient source code management by maintaining complete project history and supporting collaboration.
- **Cloud Hosting:** AWS EC2 provides a scalable and reliable environment for hosting web applications with high availability.
- **Security:** SSH authentication, SCP file transfer, and GitHub Secrets ensure secure communication and protect sensitive deployment credentials.
- **Scalability:** The architecture can be extended to support larger applications, multiple environments, and containerized deployments.
- **Reliability:** Automated workflows ensure consistent deployment every time code is updated, minimizing configuration errors.
- **Cost Efficiency:** GitHub Actions offers generous free usage limits for many projects, while AWS EC2 allows flexible resource allocation based on application requirements.

Technology Stack Architecture

The project follows a layered technology architecture where each layer performs a dedicated responsibility.

Development Layer

- Visual Studio Code
- HTML
- CSS
- JavaScript

This layer is responsible for designing and developing the hospital website.

Version Control Layer

- Git
- GitHub

This layer manages source code, maintains version history, and stores the project in a centralized repository.

Automation Layer

- GitHub Actions

This layer continuously monitors the repository for changes and automatically executes the deployment workflow.

Cloud Infrastructure Layer

- AWS EC2
- Ubuntu Linux

This layer provides the cloud-based virtual server where the website is deployed and executed.

Web Server Layer

- Apache HTTP Server

This layer hosts the website and responds to user requests by serving web pages through HTTP.

Security Layer

- SSH
- SCP
- GitHub Secrets

This layer secures authentication, encrypts communication, and safely transfers deployment files between GitHub Actions and AWS EC2.

Why This Technology Stack Was Chosen

The technologies used in this project were selected because they are widely adopted in modern software development and DevOps environments. Git and GitHub provide robust version control and collaboration features, while GitHub Actions simplifies the implementation of CI/CD without requiring additional third-party tools. AWS EC2 offers flexible cloud infrastructure that supports reliable website hosting, and Apache is one of the most trusted and widely used web servers for serving static websites.

Furthermore, SSH and SCP provide secure remote access and encrypted file transfer, ensuring that deployment activities are performed safely. Together, these technologies create an end-to-end automated deployment pipeline that is efficient, secure, scalable, and aligned with industry best practices.

Overall Technology Workflow

The overall technology workflow implemented in this project can be summarized as follows:

1. The developer creates or updates the website using HTML, CSS, and JavaScript in Visual Studio Code.
2. The changes are tracked and committed using Git.
3. The updated source code is pushed to the GitHub repository.
4. GitHub Actions automatically detects the push event.
5. The workflow securely authenticates with the AWS EC2 server using SSH credentials stored as GitHub Secrets.
6. Updated website files are transferred to the server using SCP.
7. Apache replaces the existing website files with the latest version.
8. The Apache service is restarted.
9. The updated hospital website becomes immediately accessible through the public IP address of the AWS EC2 instance.

This workflow demonstrates a complete Continuous Integration and Continuous Deployment (CI/CD) pipeline that ensures faster, reliable, and fully automated website deployment with minimal manual intervention.

4. SYSTEM DESIGN / ARCHITECTURE

Overview

The system architecture of the proposed project is designed to automate the deployment of a static hospital website using Continuous Integration and Continuous Deployment (CI/CD) practices. The architecture integrates multiple technologies, including Git, GitHub, GitHub Actions, AWS EC2, Apache HTTP Server, SSH, and SCP, to establish a secure and automated deployment pipeline.

The entire deployment process begins when the developer modifies the website source code and pushes the changes to the GitHub repository. GitHub Actions continuously monitors the repository for updates and automatically triggers the deployment workflow whenever changes are detected in the main branch. The workflow securely connects to the AWS EC2 server using SSH authentication, transfers the updated website files using the Secure Copy Protocol (SCP), replaces the existing website files, restarts the Apache web server, and publishes the latest version of the website without requiring any manual intervention.

The architecture is modular, scalable, and secure, ensuring reliable software delivery while significantly reducing deployment time and eliminating human errors associated with manual deployment processes.

Architecture Components

1. Developer Environment

The development environment consists of Visual Studio Code, where developers design and modify the hospital website using HTML, CSS, and JavaScript. Git is integrated with the development environment to track code changes, maintain version history, and facilitate collaborative development.

Responsibilities

- Develop website pages.
- Modify existing source code.
- Commit code changes using Git.
- Push updates to the GitHub repository.

2. GitHub Repository

GitHub acts as the centralized source code repository where all project files are stored. It maintains the complete version history of the application and serves as the trigger point for the CI/CD pipeline.

Responsibilities

- Store source code securely.
- Manage version history.
- Support collaborative development.
- Trigger GitHub Actions workflows.

3. GitHub Actions (CI/CD Engine)

GitHub Actions is the automation engine responsible for executing the deployment workflow. It continuously listens for push events on the repository. Once triggered, it performs all deployment activities automatically.

The workflow performs the following tasks:

- Checks out the latest source code.
- Reads deployment credentials from GitHub Secrets.
- Establishes a secure SSH connection.
- Transfers website files using SCP.
- Executes deployment commands.
- Restarts the Apache service.
- Reports workflow execution status.

This automation completely eliminates the need for manual deployment.

4. Secure Communication Layer

Security is one of the most important aspects of the deployment architecture.

The communication between GitHub Actions and AWS EC2 is established using SSH key authentication. Instead of passwords, encrypted SSH private keys stored in GitHub Secrets are used for authentication.

SCP is used for encrypted file transfer between GitHub Actions and the cloud server, ensuring confidentiality and integrity of deployment files.

Security mechanisms include:

- SSH Key Authentication
- GitHub Secrets
- Secure Copy Protocol (SCP)
- Restricted EC2 Security Groups

5. AWS EC2 Server

AWS Elastic Compute Cloud (EC2) provides the virtual machine where the hospital website is hosted.

The EC2 instance runs Ubuntu Linux and contains Apache HTTP Server. All website files are deployed to the Apache document root directory (/var/www/html).

Whenever deployment occurs, the old website files are replaced with the latest version, ensuring that visitors always access the newest content.

6. Apache HTTP Server

Apache is responsible for serving the website to end users.

After deployment, Apache automatically loads the newly deployed files and serves them over HTTP. Restarting the Apache service ensures that the updated website becomes immediately available.

Apache provides:

- Fast content delivery
- Reliable web hosting
- Easy configuration
- Compatibility with Ubuntu Linux

7. End Users

End users access the deployed hospital website using any standard web browser.

Since deployment is fully automated, users always interact with the latest version of the website without experiencing deployment delays or downtime.

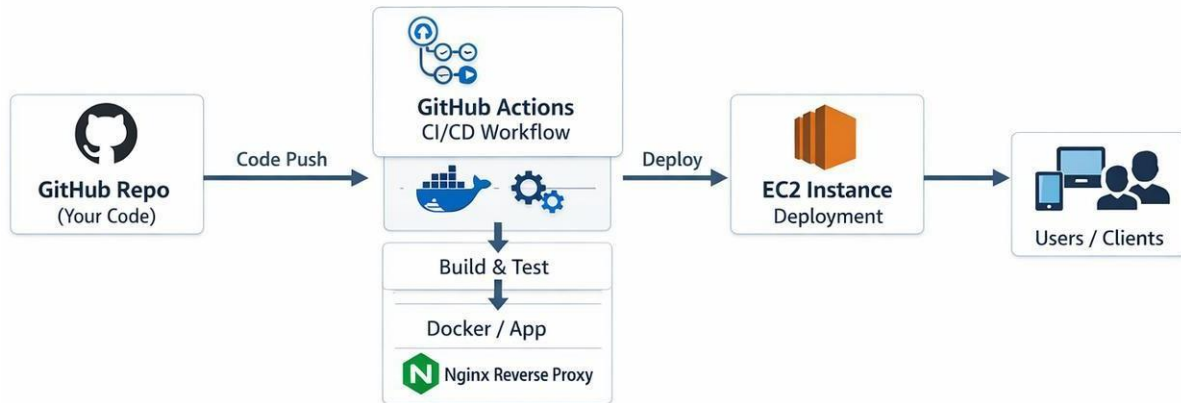
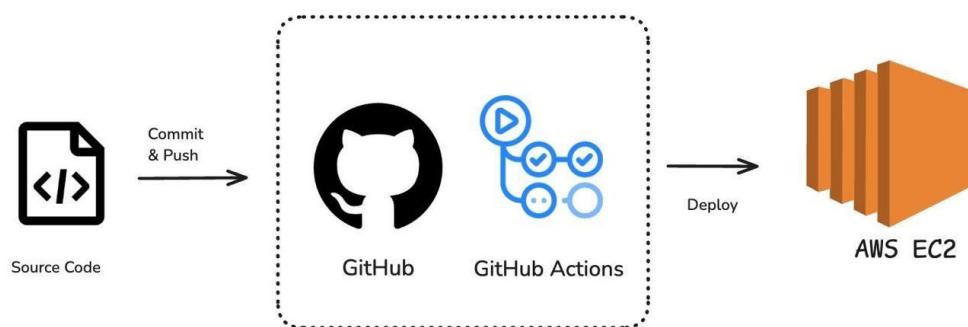


Fig. Overall System Architecture of the Proposed CI/CD System



Overall System Workflow

The architecture follows the sequence below:

1. Developer updates the website source code.
2. Changes are committed using Git.
3. Code is pushed to the GitHub repository.
4. GitHub detects the push event.
5. GitHub Actions automatically starts the CI/CD workflow.
6. Workflow authenticates with AWS EC2 using SSH.
7. Updated website files are transferred through SCP.
8. Existing files inside /var/www/html are replaced.
9. Apache HTTP Server is restarted.
10. Updated website becomes live for end users.

This workflow creates a completely automated deployment pipeline that minimizes manual effort while ensuring reliable software delivery.

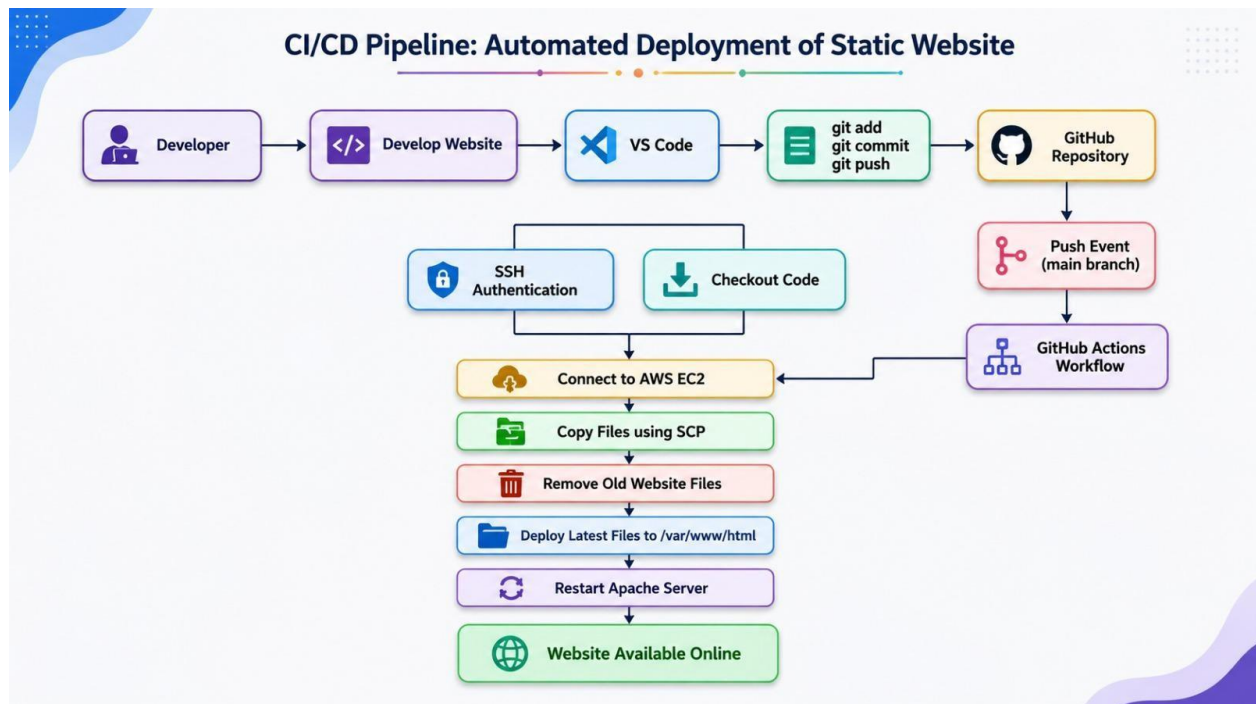


Fig. Architecture

Advantages of the Proposed Architecture

The proposed architecture offers several significant advantages:

- Fully automated deployment process.
- Reduced deployment time.
- Elimination of manual configuration errors.
- Secure authentication using SSH keys.
- Reliable cloud hosting through AWS EC2.
- Continuous delivery of updated website versions.
- Easy maintenance and scalability.
- Improved developer productivity.
- Faster software release cycles.
- Alignment with modern DevOps practices.

Architectural Features

The proposed system architecture possesses the following characteristics:

- Modular design
- Automated CI/CD workflow
- Cloud-based infrastructure
- Secure communication
- Scalable deployment process
- High reliability
- Easy maintenance
- Centralized version control
- Minimal human intervention
- Industry-standard DevOps implementation

The architecture successfully combines cloud computing, automation, version control, and secure deployment technologies into a unified system. It ensures that every code modification is automatically validated, transferred, deployed, and made available to users, thereby improving deployment efficiency, consistency, and overall software quality.

4. IMPLEMENTATION

Overview

The implementation phase focuses on the practical execution of the proposed CI/CD pipeline for automatically deploying a static hospital website to an AWS EC2 instance. The project integrates web development technologies, version control, cloud computing, and automation tools to establish a secure and efficient deployment workflow. The implementation was carried out in multiple stages, beginning with website development and ending with automated deployment using GitHub Actions.

1. Website Development

The hospital website was developed as a static web application using HTML, CSS, and JavaScript. HTML was used to create the structure of the web pages, CSS was used to design an attractive and responsive user interface, and JavaScript was used to implement interactive features and improve user experience.

The website includes multiple pages containing hospital-related information such as:

- Home Page
- About Us
- Services
- Doctors
- Departments
- Contact Information
- Appointment Section

The source code was developed and maintained using Visual Studio Code, providing an efficient environment for coding, debugging, and project management.

2. Version Control using Git

Git was used to manage the complete source code throughout the development process. Every modification made to the website was tracked using Git, allowing developers to maintain version history and recover previous versions whenever required.

The implementation followed the standard Git workflow:

- Initialize the Git repository.
- Add project files.
- Commit changes with appropriate commit messages.
- Push the latest code to the GitHub repository.

This approach ensured proper version management and simplified collaboration.

3. GitHub Repository Configuration

The project source code was uploaded to a GitHub repository, which served as the central platform for source code management.

The repository performs several important functions:

- Stores all website files securely.
- Maintains complete commit history.
- Enables collaboration.
- Triggers the CI/CD workflow whenever changes are pushed to the main branch.

GitHub acts as the starting point of the automated deployment pipeline.

4. AWS EC2 Instance Setup

An Amazon EC2 virtual machine running Ubuntu Linux was created to host the hospital

website.

The following configuration steps were performed:

- Launch an Ubuntu EC2 instance.
- Configure Security Groups to allow HTTP (Port 80) and SSH (Port 22) access.
- Connect to the server using SSH.
- Update system packages.
- Install Apache HTTP Server.
- Enable and start the Apache service.
- Verify successful installation by accessing the server through its public IP address.

Apache was configured to serve website files from the default document root directory:

`/var/www/html`

This server acts as the production environment where the website is hosted.

5. SSH Key Generation and GitHub Secrets Configuration

Secure authentication was implemented using SSH key-based authentication.

The implementation involved the following steps:

- Generate an SSH public-private key pair.
- Copy the public key to the EC2 server.
- Store the private SSH key securely in GitHub Secrets.
- Save additional deployment parameters such as:
 - EC2 Host
 - EC2 Username
 - SSH Private Key

Using GitHub Secrets prevents sensitive credentials from being exposed in the source code and ensures secure communication between GitHub Actions and AWS EC2.

6. GitHub Actions Workflow Implementation

The automation workflow was implemented using GitHub Actions.

A YAML workflow file was created inside the `.github/workflows` directory of the repository.

The workflow performs the following operations automatically:

- Detects code push events.
- Checks out the latest repository.
- Authenticates with AWS EC2.
- Transfers updated website files using SCP.
- Executes deployment commands through SSH.
- Restarts the Apache web server.
- Reports deployment status.

As a result, every code push automatically deploys the latest website version without requiring manual intervention.

7. Automated Deployment Process

After successful workflow execution, the deployment process performs the following operations:

- Existing website files are removed.
- Updated project files are copied to `/var/www/html`.

- Apache HTTP Server is restarted.
- Latest website becomes accessible through the EC2 public IP address.

This automation significantly reduces deployment time and eliminates repetitive manual deployment tasks.

8. Testing and Verification

The implementation was verified through multiple deployment tests.

Testing activities included:

- Verifying successful Git commits and pushes.
- Monitoring GitHub Actions workflow execution.
- Confirming secure SSH authentication.
- Checking successful file transfer using SCP.
- Verifying Apache restart after deployment.
- Accessing the website through the EC2 public IP.
- Ensuring that newly added changes were immediately reflected on the live website.

The successful execution of these tests confirmed that the automated deployment pipeline functioned correctly and consistently.

9. Final Deployment Workflow

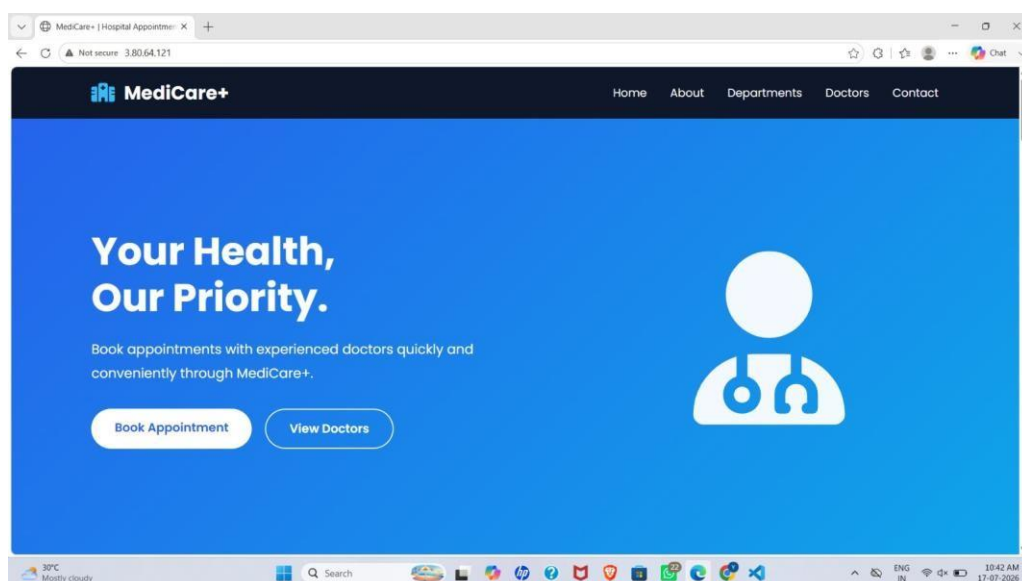
The complete implementation follows the sequence below:

1. Developer modifies website source code.
2. Changes are committed using Git.

3. Code is pushed to GitHub.
4. GitHub Actions automatically detects the push.
5. Workflow authenticates with AWS EC2 using SSH.
6. Updated website files are transferred using SCP.
7. Existing files on the server are replaced.
8. Apache HTTP Server is restarted.
9. Updated website becomes live and accessible to users.

Implementation Outcome

The implementation successfully achieved the objectives of the project by automating the deployment of a static hospital website using GitHub Actions and AWS EC2. The developed CI/CD pipeline eliminated manual deployment procedures, improved deployment speed, ensured secure communication through SSH authentication, and provided a reliable cloud-based hosting environment. The integration of Git, GitHub, GitHub Actions, AWS EC2, Apache, SSH, and SCP demonstrated an industry-standard DevOps workflow capable of supporting efficient, consistent, and scalable software deployment.



6. RESULTS

Overview

The implementation of the automated deployment pipeline was successfully completed using GitHub Actions and AWS EC2. The developed CI/CD pipeline effectively automated the deployment of the static hospital website, eliminating manual deployment activities and ensuring that every code update was reflected on the live server immediately after being pushed to the GitHub repository. The project achieved all the planned objectives by integrating version control, workflow automation, cloud hosting, and secure deployment into a single automated system. The deployment process was tested multiple times by modifying the website source code and verifying that the changes were automatically deployed to the AWS EC2 server without requiring manual intervention.

1. Successful Website Deployment

The static hospital website was successfully hosted on an AWS EC2 instance running the Ubuntu operating system. Apache HTTP Server served the website efficiently, making it accessible through the public IP address of the EC2 instance.

The deployment confirmed that the web server was properly configured and capable of serving all website pages without errors. The website remained accessible after every automated deployment, demonstrating the stability and reliability of the hosting environment.

2. Successful GitHub Actions Workflow Execution

The GitHub Actions workflow executed successfully whenever code changes were pushed to the main branch of the repository. The workflow automatically performed repository checkout, server authentication, secure file transfer, deployment, and Apache restart. The workflow logs confirmed that each deployment stage completed successfully without any manual intervention, proving the effectiveness of the implemented CI/CD pipeline.

3. Secure File Transfer and Deployment

SSH key authentication enabled secure communication between GitHub Actions and the AWS EC2 instance. Website files were transferred securely using the Secure Copy Protocol (SCP), ensuring that sensitive deployment data remained protected.

The deployment process successfully replaced the previous website files inside the Apache document root directory (/var/www/html) and restarted the Apache service automatically.

4. Automatic Website Update

One of the major achievements of this project is the successful implementation of Continuous Deployment. Whenever modifications were made to the website and pushed to the GitHub repository, GitHub Actions automatically deployed the latest version to the cloud server.

The deployment process required no manual login, file upload, or server configuration. The updated website became available online within a short period after each code push, significantly reducing deployment time.

5. Performance and Reliability

The automated deployment pipeline improved the overall efficiency of the deployment process by eliminating repetitive manual tasks and reducing the possibility of deployment errors.

The system demonstrated:

- Faster deployment execution.
- Consistent deployment results.
- Secure communication using SSH authentication.
- Reliable website hosting on AWS EC2.
- Minimal deployment downtime.
- Automated deployment with no manual intervention.

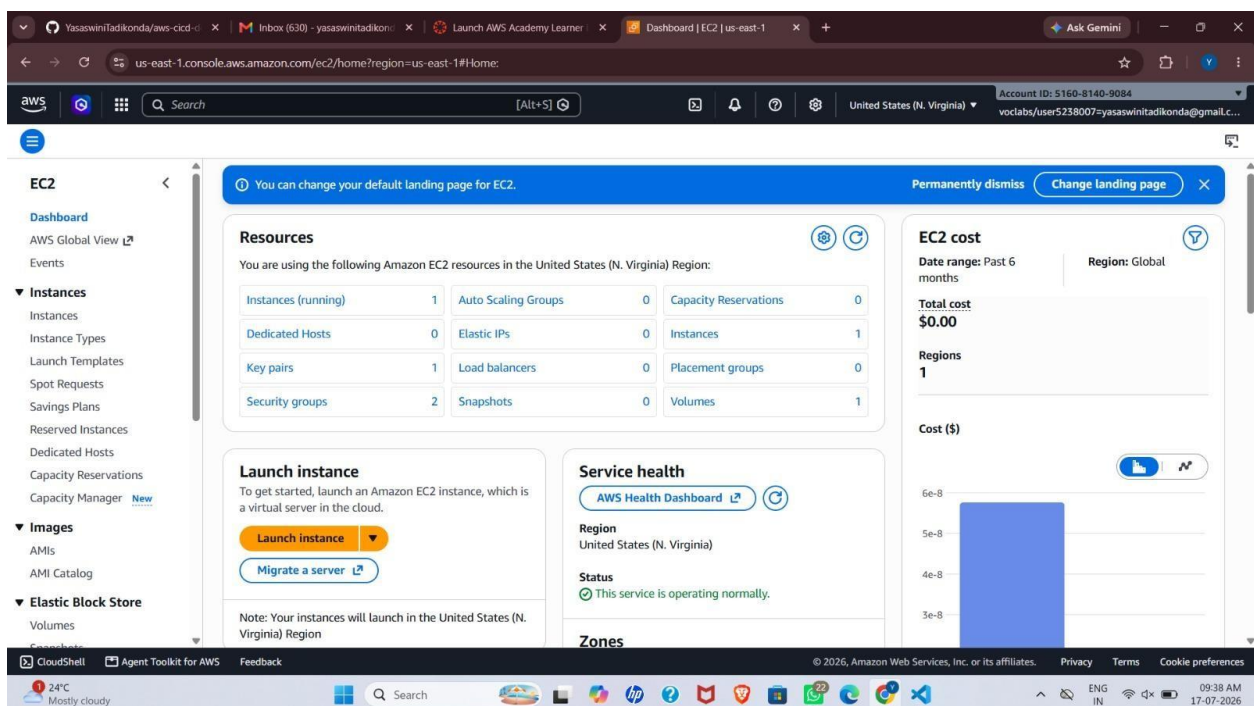
Multiple deployment tests confirmed that the pipeline functioned consistently and accurately across repeated executions.

Project Outcomes

- Developed a responsive static hospital website.
- Hosted the website on AWS EC2 using Apache HTTP Server.
- Implemented secure SSH-based authentication.
- Automated website deployment using GitHub Actions.
- Established a complete CI/CD pipeline.
- Improved deployment speed and consistency.
- Reduced human errors during deployment.
- Demonstrated practical implementation of DevOps principles.

Result Summary

The project successfully demonstrates the practical implementation of a Continuous Integration and Continuous Deployment (CI/CD) pipeline for automated website deployment. The integration of Git, GitHub, GitHub Actions, AWS EC2, Apache HTTP Server, SSH, and SCP resulted in a secure, reliable, and efficient deployment system. The automated pipeline significantly reduced deployment time, ensured consistency across deployments, and minimized manual effort. Overall, the developed system validates the effectiveness of DevOps practices in modern cloud-based application deployment and provides a scalable foundation for future enhancements.



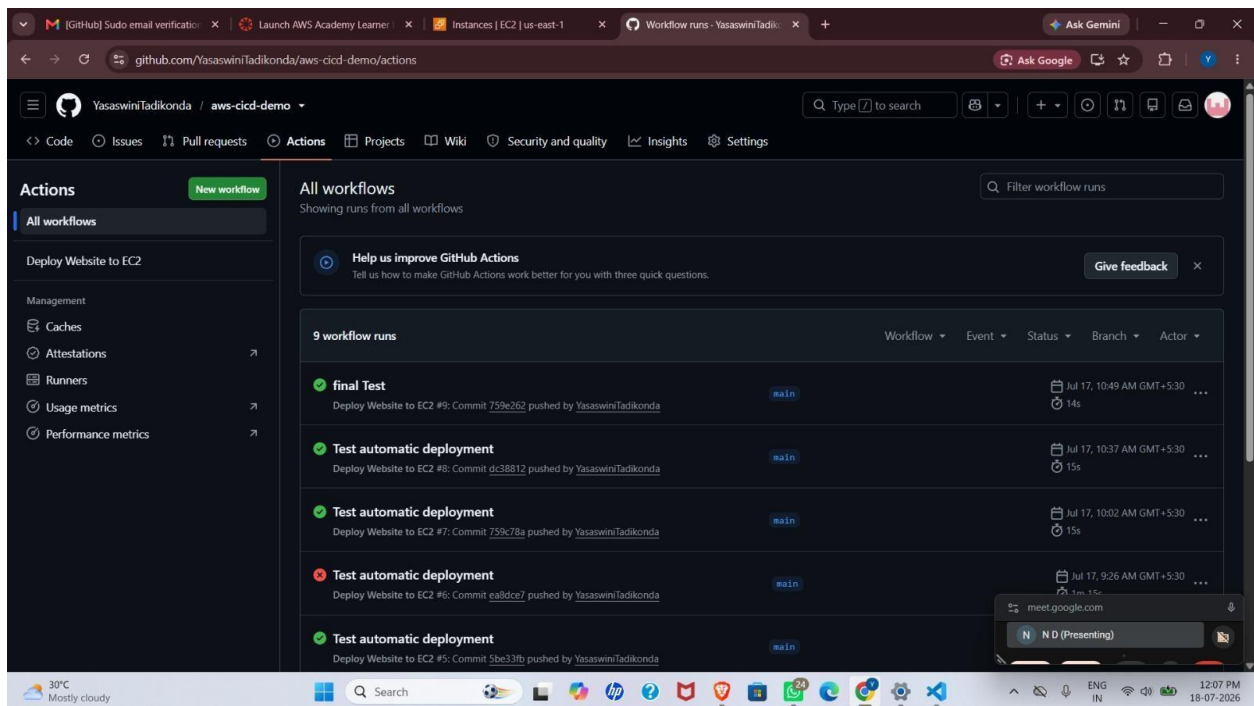


Fig. Successful Execution of GitHub Actions CI/CD Workflow

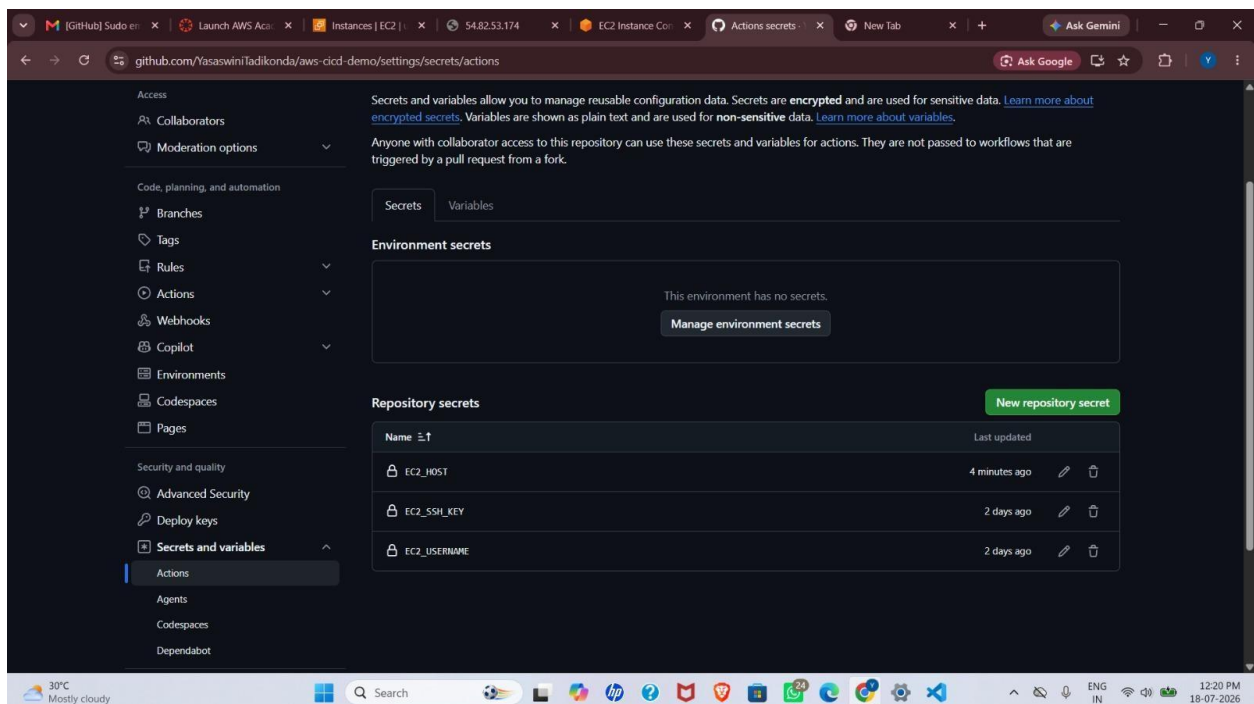


Fig. Automated Website Deployment Triggered by GitHub Actions

7. CONCLUSION

The successful implementation of this project demonstrates the effectiveness of Continuous Integration and Continuous Deployment (CI/CD) in automating the deployment of web applications. By integrating Git, GitHub, GitHub Actions, AWS EC2, Apache HTTP Server, SSH, and SCP, the project establishes a secure, reliable, and fully automated deployment pipeline for a static hospital website. The automated workflow eliminates the need for manual deployment, thereby reducing human errors, minimizing deployment time, and ensuring that the latest version of the website is consistently available to users.

The project successfully achieved its primary objectives by developing a static hospital website, hosting it on an AWS EC2 instance, and automating the deployment process using GitHub Actions. Every code change pushed to the GitHub repository automatically triggered the deployment workflow, securely transferred the updated files to the cloud server, replaced the existing website content, and restarted the Apache web server without requiring any manual intervention. The successful execution of multiple deployment cycles confirmed the reliability, consistency, and efficiency of the implemented CI/CD pipeline.

In addition to achieving technical objectives, this project provided valuable practical experience in modern DevOps methodologies, cloud computing, version control, workflow automation, and secure server management. The implementation closely reflects real-world software development practices followed by organizations for rapid and reliable software delivery. It also strengthened understanding of cloud infrastructure, automation tools, and deployment strategies that are widely used in the software industry.

Furthermore, the developed solution provides a strong foundation for future enhancements. The deployment pipeline can be extended to support advanced DevOps technologies such as Docker containerization, Kubernetes orchestration, HTTPS implementation using SSL/TLS certificates, load balancing, auto-scaling, monitoring with AWS CloudWatch, and multi-environment deployments for development, testing,

and production. These enhancements would improve the scalability, security, and maintainability of the system while making it suitable for enterprise-level applications.

In conclusion, the project successfully demonstrates how the integration of DevOps practices with cloud computing technologies can simplify software deployment, improve operational efficiency, and deliver reliable web applications with minimal manual effort. The implemented CI/CD pipeline serves as an effective example of modern automated deployment techniques and highlights the importance of automation in building scalable, secure, and efficient software delivery systems.